

Práctica 5: Tabla de tipos y tabla de símbolos

Compiladores 2026-2

Profesora: Ariel Adara Mercado Martínez

Alumno

Edwin Rodríguez Nevarez - 312227441

Objetivo

Implementar una tabla de tipos (TypeTab) capaz de almacenar tipos primitivos, arreglos y estructuras (registros). Implementar una pila de tablas de símbolos (PilaTs) que permita manejar alcances (scopes) durante el análisis semántico. Integrar las reglas semánticas de declaración de variables en un esquema de traducción dirigido por la sintaxis (ETDS) utilizando Flex y Bison. Comprender cómo se llena la tabla de tipos y la pila de tablas de símbolos durante la declaración de variables, tipos arreglo y tipos registro (struct).

Desarrollo

Ejercicio 1: Completar TypeTab

Se completaron los métodos `addArrayType` y `addStructType` en el archivo `TypeTab.cpp`.

Para los arreglos, primero calculamos la memoria total multiplicando el número de elementos por lo que pesa el tipo base (que obtenemos con la función `getTam`). Después, simplemente guardamos este nuevo tipo en el mapa de C++ usando el contador `nextId` y lo incrementamos para que el siguiente tipo no choque con este. Para los structs hicimos lo mismo, pero la función ya nos pasa el tamaño total, así que solo tuvimos que guardarlo.

```
1 int TypeTab::addArrayType(int numElems, int tipoBase) {
2     int tam = numElems * getTam(tipoBase);
3     types[nextId] = Type("array", tam, numElems, tipoBase);
4     int idAsignado = nextId;
5     nextId++;
6     return idAsignado;
7 }
8
9 int TypeTab::addStructType(int tam) {
10    types[nextId] = Type("struct", tam);
11    int idAsignado = nextId;
12    nextId++;
13    return idAsignado;
14 }
```

Ejercicio 2: Completar SymTab

Primero se implementó la función `existe(string id)` usando la función `find()` de los mapas de C++. Con esto, buscar si una variable ya existe es muy rápido. Luego, en las

funciones `addSym`, pusimos un condicional `if` al principio: si la función `existe` nos dice que el nombre ya está ocupado, regresamos `false` para que luego el compilador marque un error. Si no existe, guardamos el símbolo nuevo en el mapa junto con su dirección de memoria, su ID de tipo y su categoría.

```
1 bool SymTab::existe(string id) {
2     return syms.find(id) != syms.end();
3 }
4
5 bool SymTab::addSym(string id, int dir, int tipo, string cat) {
6     if (existe(id)) {
7         return false;
8     }
9     syms[id] = Sym(dir, tipo, cat);
10    return true;
11 }
12
13 bool SymTab::addSym(string id, int dir, int tipo, string cat, vector<int>
14    params) {
15     if (existe(id)) {
16         return false;
17     }
18     syms[id] = Sym(dir, tipo, cat, params);
19     return true;
20 }
```

Ejercicio 3: Acciones Semánticas en `parser.yy`

En el análisis semántico se utilizó un Esquema de Traducción Dirigido por la Sintaxis.

Para heredar el tipo base en la producción recursiva de arreglos ($A \rightarrow [\text{num}] A$), se creó una variable global auxiliar `currentBaseType` que guarda el tipo definido en la producción B. Así, cuando la recursión termina en $A \rightarrow \epsilon$, el arreglo final toma correctamente el ID de su tipo primitivo.

El manejo de alcances (*scopes*) se hizo con las acciones intermedias (*mid-rule actions*) de Bison:

- **Funciones** (`def T id (F) {S}`): Justo antes de evaluar el cuerpo `S`, se guardó la dirección de memoria global actual en `pilaDir`, se empujó una nueva tabla de símbolos local a `pilaTs` y se reinició la variable `dir` a cero. Todos los parámetros recibidos por `F` y variables declaradas adentro modificaron únicamente este alcance local. Al salir de `S`, se extrajo la tabla local, se restauró la memoria global de `pilaDir` y se guardó la firma de la función en la tabla de alcance global (`pilaTs.bottom()`).
- **Estructuras** (`register {D}`): En las funciones, se abre un nuevo contexto. Después procesar sus declaraciones internas, el tamaño de memoria `dir` es el tamaño total en bytes del `struct`, el cual fue registrado como un tipo compuesto en `tablaTipos.addStructType(dir)`.
- **Variables** (`L`): Cada vez que se declaró una variable o parámetro, se insertó a `pilaTs.top()` y se incrementó `dir` obteniendo el tamaño del tipo en la `TypeTab`.

```

1 // =====
2 // EJERCICIO 3: Esquema de Traducción Dirigido por la Sintaxis
3 // =====
4
5 // P -> D (Inicialización de alcance global)
6 P : {
7     dir = 0;
8     pilaTs.push(new SymTab());
9 }
10 D
11 {
12     std::cout << std::endl;
13     tablaTipos.print();
14     std::cout << std::endl;
15     std::cout << "Tabla de simbolos global:" << std::endl;
16     pilaTs.top()->print();
17 }
18 ;
19
20 // D -> T L ; D | def T id ( F ) { S } D | epsilon
21 D : T L SEMICOLON D { }
22 | DEF T ID LPAREN F RPAREN
23 {
24     std::string id = $3;
25     if (!pilaTs.bottom()->existe(id)) {
26         pilaTs.push(new SymTab()); // Nuevo alcance local
27         pilaDir.push(dir);
28         dir = 0;
29         tipoReturnFunc = $2.tipo;
30     } else {
31         std::cerr << "Error: La funcion '" << id << "' ya fue
32         declarada." << std::endl;
33     }
34     LBRACE S RBRACE
35 {
36     std::string id = $3;
37     pilaTs.pop(); // Retirar alcance local
38     dir = pilaDir.top(); // Restaurar memoria global
39     pilaDir.pop();
40
41     pilaTs.bottom()->addSym(id, -1, $2.tipo, "func", listaParams);
42     listaParams.clear();
43     if ($3) free($3);
44 }
45 D
46 | /* epsilon */ ;
47
48 // T -> B A | register { D }
49 T : B A
50 {
51     $$.tipo = $2.tipo;
52     currentType = $$.tipo;
53 }

```

```

54 | REGISTER LBRACE
55 | {
56 |     pilaTs.push(new SymTab()); // Nuevo alcance para struct
57 |     pilaDir.push(dir);
58 |     dir = 0;
59 | }
60 | D RBRACE
61 | {
62 |     int tamStruct = dir;
63 |     pilaTs.pop();
64 |
65 |     int idNuevoStruct = tablaTipos.addStructType(tamStruct);
66 |
67 |     dir = pilaDir.top();
68 |     pilaDir.pop();
69 |
70 |     $$.tipo = idNuevoStruct;
71 |     currentType = $$.tipo;
72 | }
73 | ;
74 |
75 // B -> int | float
76 B : INT
77 | {
78 |     $$.tipo = tablaTipos.getId("int");
79 |     $$base = $$.tipo;
80 |     currentBaseType = $$.tipo; // Propagar a A
81 | }
82 | FLOAT
83 | {
84 |     $$.tipo = tablaTipos.getId("float");
85 |     $$base = $$.tipo;
86 |     currentBaseType = $$.tipo;
87 | }
88 | ;
89 |
90 // A -> [ NUM ] A | epsilon
91 A : LBRACKET NUM RBRACKET A
92 | {
93 |     if ($2 > 0) {
94 |         $$.tipo = tablaTipos.addArrayType($2, $4.tipo);
95 |     } else {
96 |         std::cerr << "Error: El indice debe ser mayor a cero." << std
97 |         ::endl;
98 |         $$.tipo = $4.tipo;
99 |     }
100 | }
101 | /* epsilon */
102 | {
103 |     $$.tipo = currentBaseType; // Herencia del tipo primitivo
104 | }
105 | ;
106 // F -> T id , F | T id | epsilon

```

```

107 F : T ID COMMA F
108 {
109     std::string id = $2;
110     pilaTs.top()->addSym(id, dir, $1.tipo, "param");
111     dir += tablaTipos.getTam($1.tipo);
112     listaParams.push_back($1.tipo);
113     if ($2) free($2);
114 }
115 | T ID
116 {
117     std::string id = $2;
118     pilaTs.top()->addSym(id, dir, $1.tipo, "param");
119     dir += tablaTipos.getTam($1.tipo);
120     listaParams.push_back($1.tipo);
121     if ($2) free($2);
122 }
123 | /* epsilon */ ;
124
125 // S -> epsilon (cuerpo vacio para esta practica)
126 S : /* epsilon */ ;
127
128 // L -> L , id | id
129 L : L COMMA ID
130 {
131     std::string id = $3;
132     if (!pilaTs.top()->existe(id)) {
133         pilaTs.top()->addSym(id, dir, currentType, "var");
134         dir += tablaTipos.getTam(currentType);
135     } else {
136         std::cerr << "Error: Variable '" << id << "' ya declarada." <<
std::endl;
137     }
138     if ($3) free($3);
139 }
140 | ID
141 {
142     std::string id = $1;
143     if (!pilaTs.top()->existe(id)) {
144         pilaTs.top()->addSym(id, dir, currentType, "var");
145         dir += tablaTipos.getTam(currentType);
146     } else {
147         std::cerr << "Error: Variable '" << id << "' ya declarada." <<
std::endl;
148     }
149     if ($1) free($1);
150 }
151 ;

```

Ejercicio 4: Prueba

Para checar que funcione bien el ETDS, se hicieron dos archivos de prueba con todos los posibles escenarios.

Archivo: prueba_valida Este archivo tiene declaración de variables simples, el cálculo de memoria para arreglos, la anidación de un entorno local para un **struct** y el correcto registro de una función con parámetros en la tabla de símbolos global.

```
1 int a, b;
2 float c;
3 int [5] arreglo;
4
5 register {
6     int x;
7     float y;
8 } punto;
9
10 def int miFuncion (int param1, float param2) {
11 }
```

Archivo: prueba_invalida Este archivo es para checar la validación de la Tabla de Símbolos, intentando redeclarar una variable dos veces.

```
1 int a;
2 float a;
```

Ejercicio 5: Resultados

Al terminar el análisis, se imprime la Tabla de Tipos y la Tabla de Símbolos global.

- Hace el tipo **array** de 20 bytes (5 elementos $\times$ 4 bytes del tipo base **int**).
- Hace el tipo **struct** con un tamaño total de 8 bytes.
- Calcula las direcciones de memoria (**dir**) para las variables globales **a**, **b** y **c**.
- Atrapa el error semántico de redeclaración.

```
edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P5/src$ ./comp prueba_valida
Analizando archivo: prueba_valida
-----
===== TABLA DE TIPOS =====
ID   Nombre   Tam   Elems   Base
----
0    int       4     -1      -1
1    float     4     -1      -1
2    array     20    5        0
3    struct    8     -1      -1
=====

Tabla de simbolos global:
===== TABLA DE SIMBOLOS =====
Nombre   Dir   Tipo   Cat
-----
a         0     0      var
arreglo   12    2      var
b         4     0      var
c         8     1      var
miFuncion -1     0      func
param1    44    0      param
param2    40    1      param
punto     32    3      var
=====

-----
Análisis completado exitosamente.
edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P5/src$ ./comp prueba_invalida
Analizando archivo: prueba_invalida
-----
Error: La variable 'a' ya fue declarada en este alcance.

===== TABLA DE TIPOS =====
ID   Nombre   Tam   Elems   Base
----
0    int       4     -1      -1
1    float     4     -1      -1
=====

Tabla de simbolos global:
===== TABLA DE SIMBOLOS =====
Nombre   Dir   Tipo   Cat
-----
a         0     0      var
=====

-----
Análisis completado exitosamente.
edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P5/src$
```

Figura 1: Ejecución del compilador mostrando la Tabla de Tipos, la Tabla de Símbolos global y la detección de errores semánticos.